

COMPLEX COMPUTING ACTIVITY

Can AI Save Lives?

Building an Intelligent Clinical Early Warning System

Section 1: Understanding the Clinical Prediction Problem

Why ICU Risk Prediction Is Difficult

Predicting patient deterioration in an ICU environment is significantly more complex than ordinary machine learning classification problems. In tasks such as spam detection, fraud analysis, or image recognition, the input data is generally stable, the categories are clearly separated, and prediction mistakes usually have manageable consequences. In healthcare, especially in critical care units, these assumptions do not apply. A patient's health status can change rapidly, the difference between stable and unstable conditions is often unclear, and an incorrect prediction may lead to delayed treatment or loss of life.

The dataset used in this project, the Patient Survival Prediction dataset from Kaggle, contains information from 64,199 ICU admissions. One major challenge within the dataset is severe class imbalance. Only 8.6% of patients died, while the remaining majority survived. Because of this imbalance, a model could predict "survived" for every patient and still achieve over 91% accuracy. Although this result appears statistically strong, such a model would fail to identify any critically ill patients and would therefore be useless in a real clinical setting. This demonstrates why healthcare prediction systems require more meaningful evaluation metrics than accuracy alone.

Another important challenge is that deterioration cannot usually be identified from a single vital sign. Physiological measurements naturally vary over time. For example, an elevated heart rate may result from pain, stress, medication, or physical activity rather than severe illness. Clinicians therefore focus on trends and combinations of measurements instead of isolated values. Understanding how multiple signals evolve together over time is essential for detecting patient risk accurately.

Limitations of Numerical Vital Signs

Structured medical datasets mainly contain numerical measurements such as heart rate, respiratory rate, blood pressure, body temperature, and oxygen saturation (SpO2). While these features are clinically valuable, they do not fully represent the patient's condition. The same

numerical value can have completely different meanings depending on patient history, diagnosis, and overall health status. For example, a heart rate of 120 bpm may be acceptable for a young recovering patient but highly concerning for an elderly patient with sepsis or organ dysfunction.

Clinical notes and nursing observations provide important contextual information that numbers alone cannot capture. Physicians often document symptoms, behavioural changes, medical history, treatment response, and overall clinical impressions in free-text form. A statement such as “patient becoming increasingly confused despite stable vitals” may indicate serious deterioration even when standard measurements appear normal. These details are often critical for accurate clinical decision-making.

Because the dataset used in this project did not contain real physician notes, structured patient information was converted into clinical-style narratives. Demographics, diagnosis systems, APACHE scores, comorbidities, and vital-sign trends were combined into natural-language descriptions. This technique is commonly used in clinical NLP research and allowed the project to evaluate Transformer-based language models such as ClinicalBERT using generated medical narratives.

Importance of Recall in Healthcare AI

In binary classification, Recall measures how many actual positive cases are correctly identified by the model. In this project, positive cases represent patients who are at risk of deterioration or death. Recall is calculated as:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

where TP refers to true positives and FN refers to false negatives. A false negative occurs when the system incorrectly predicts that a critically ill patient is stable.

In healthcare systems, Recall is often more important than Accuracy because the consequences of missing a dangerous case are extremely serious. Consider a patient admitted to the ICU after emergency surgery. Over several hours, the patient’s heart rate rises gradually, blood pressure begins to fall, and oxygen saturation slowly decreases. Individually, these values may appear only slightly abnormal, but together they may indicate the onset of septic shock.

If the model incorrectly predicts that the patient is stable, no warning will be triggered and treatment may be delayed. In a real ICU environment, such a delay could significantly worsen the patient’s condition before medical staff intervene. For this reason, early warning systems must prioritise detecting as many high-risk patients as possible, even if that increases the number of false alarms. In clinical AI, missing a dangerous patient is generally far more harmful than generating an unnecessary alert. If the model produces a false positive, flagging her as at-risk when she is stable, a physician is called unnecessarily. The physician assesses her, finds her stable, and returns to their duties. The cost of this error is a few minutes of a clinician's time and mild disruption. This is recoverable.

This asymmetry, irreversible harm from false negatives versus recoverable inconvenience from false positives, is why every model in this project was evaluated with Recall as the primary

clinical metric. An early warning system that achieves 95% accuracy but misses 60% of deteriorating patients is not a safety tool; it is a liability. The goal is to catch as many true deteriorations as possible, even at the cost of some additional false alarms.

Section 2: Core Deep Learning Design Choices

Understanding the Vanishing Gradient Problem

Deep neural networks learn using backpropagation, where gradients of the loss function are calculated and passed backward through the layers to update model parameters. In very deep networks, these gradients are repeatedly multiplied across many layers. When activation functions such as sigmoid or tanh are used, their derivatives remain very small, usually between 0 and 0.25. As a result, gradients gradually shrink while moving backward through the network. This phenomenon is known as the vanishing gradient problem.

When gradients become extremely small, the earlier layers of the network stop learning effectively. The model then struggles to capture complex hierarchical patterns from the data, reducing overall performance.

To address this issue, all hidden layers in this project used the ReLU (Rectified Linear Unit) activation function:

$$f(x) = \max(0, x)$$

Unlike sigmoid or tanh, ReLU maintains a gradient of 1 for positive values, allowing gradients to flow more effectively during backpropagation. This helps deep networks train faster and more reliably. In addition, He Normal Initialisation was applied to the network weights because it is specifically designed for ReLU-based architectures and helps stabilise activation values during training.

Batch Normalisation was also added after each hidden layer. During training, the distribution of activations inside the network continuously changes as weights are updated, a problem called internal covariate shift. Batch Normalisation reduces this instability by normalising activations within each mini-batch. This improves gradient flow, supports higher learning rates, speeds up convergence, and provides mild regularisation against overfitting.

Experimental results confirmed its importance. Models trained without Batch Normalisation showed higher validation loss and less stable optimisation behaviour, demonstrating that BatchNorm played a significant role in improving training stability.

Comparing Adam and SGD Optimisers

Two optimisation algorithms were evaluated in this project under identical architectural settings: Adam and SGD with Momentum.

Adam (Adaptive Moment Estimation) adjusts learning rates individually for each parameter by combining gradient momentum and adaptive variance scaling. This allows the optimiser to converge quickly and efficiently, especially in deep learning problems with complex optimisation landscapes. Using a learning rate of 0.001, Adam achieved stable convergence with strong overall performance, producing an Accuracy of 0.8618 and an F1-Score of 0.4189.

SGD with Momentum used a learning rate of 0.01 and momentum value of 0.9. Instead of adapting learning rates automatically, SGD updates weights using accumulated gradient history, which helps the optimiser move more consistently across the loss surface and avoid shallow local minima. Although SGD required more training epochs and produced lower overall Accuracy (0.8301), it achieved higher Recall (0.6583), indicating stronger sensitivity toward the minority mortality class.

This comparison highlights an important clinical trade-off. Adam produced more stable training and better balanced performance overall, while SGD showed improved ability to identify high-risk patients. Since Recall is especially important in healthcare prediction tasks, both optimisation approaches provided valuable insights into model behaviour.

Difference Between Loss Function and Cost Function

The terms loss function and cost function are closely related but not identical. A loss function measures the prediction error for a single training sample. In this project, Binary Cross-Entropy Loss was used:

$$L = -(y \log(p) + (1-y) \log(1-p))$$

where y represents the true label and p represents the predicted probability.

The cost function, on the other hand, represents the average loss across all samples in a batch or the entire dataset. During training, the optimiser attempts to minimise the cost function rather than the loss of an individual sample.

Techniques such as Batch Normalisation and Dropout improve the shape of the optimisation landscape, making training smoother and helping the network generalise more effectively to unseen data.

Dropout for Reducing Overfitting

Dropout is a regularisation technique that randomly disables a fraction of neurons during training. In this project, a dropout rate of 0.3 was used. By randomly removing neurons during each training step, the network is forced to learn distributed feature representations instead of depending heavily on a small set of neurons.

During inference, all neurons remain active, but their outputs are scaled appropriately. This creates an effect similar to combining predictions from multiple smaller networks and helps improve generalisation performance.

The ablation study demonstrated the value of Dropout clearly. Models trained without Dropout showed signs of overfitting, particularly in the validation loss curves where training and validation performance diverged more quickly. Although the no-dropout model achieved comparable Accuracy, its generalisation behaviour was weaker.

In healthcare AI systems, overfitting is especially dangerous because hospital populations and clinical conditions may differ from the training dataset. A model that memorises training patterns instead of learning general clinical behaviour may perform poorly on real patients. Therefore, regularisation techniques such as Dropout are important not only for statistical performance but also for patient safety and model reliability.

Generation 1 Results Summary

Model	Accuracy	Precision	Recall ★	F1-Score
DNN (Adam)	0.8618	0.3317	0.5681	0.4189
DNN (SGD)	0.8301	0.2921	0.6583	0.4046
DNN (No Dropout)	0.8463	0.3043	0.5849	0.4003
DNN (No BatchNorm)	0.8292	0.2892	0.6499	0.4003

Section 3: Learning Patient Trends Over Time

Challenges of Standard Recurrent Neural Networks

Traditional Recurrent Neural Networks (RNNs) are designed to process sequential data by passing information from one timestep to the next. In a standard RNN, the hidden state at time step t depends on both the current input and the previous hidden state. During training, gradients are propagated backward through every timestep using a process called Backpropagation Through Time (BPTT).

A major limitation of standard RNNs is the vanishing gradient problem. Since activation functions such as tanh produce small derivatives, gradients become smaller after repeated multiplication across long sequences. Eventually, the gradients approach zero, preventing the network from learning information from earlier timesteps. This becomes a serious issue in healthcare prediction tasks because patient deterioration often develops gradually over hours or days rather than instantly.

As a result, standard RNNs struggle to capture long-term dependencies within patient vital-sign sequences, limiting their usefulness for ICU monitoring systems.

How LSTM and GRU Improve Sequence Learning

Long Short-Term Memory (LSTM) networks were developed to solve the vanishing gradient problem found in standard RNNs. LSTMs introduce a dedicated memory structure called the cell state, which allows important information to flow through the sequence with minimal loss. The architecture also includes three gates that regulate information flow:

- Forget Gate → decides which old information should be removed
- Input Gate → decides which new information should be stored
- Output Gate → controls what information is passed to the next hidden state

Because the cell state relies more on addition than repeated multiplication, gradients can move backward through long sequences more effectively. This enables LSTMs to learn long-term temporal patterns, which is extremely important in clinical monitoring where patient conditions evolve over time.

The Gated Recurrent Unit (GRU) is a simplified version of the LSTM architecture. GRUs combine the forget and input gates into a single update gate and remove the separate cell state entirely. This reduces architectural complexity and lowers the number of trainable parameters.

In the experiments conducted for this project, the GRU achieved the highest Recall score among all sequential models (0.7799) while maintaining efficient training performance. Although the GRU required slightly more training time than the standard LSTM, it provided stronger sensitivity toward high-risk patients with a simpler architecture. For real-time healthcare systems where inference speed and computational efficiency are important, GRU offers a strong balance between performance and deployment practicality.

Real-Time Monitoring vs Bidirectional Learning

Bidirectional LSTM (Bi-LSTM) models process sequences in two directions simultaneously:

- Forward direction → past to future
- Backward direction → future to past

The outputs from both directions are combined, allowing the model to learn from both previous and future context at the same time. This approach improved offline evaluation performance and produced the highest Accuracy among the sequence models tested.

However, Bidirectional LSTMs are not suitable for real-time ICU deployment. In a live monitoring environment, future patient measurements are not yet available. A Bi-LSTM would require access to future timesteps before generating a prediction, which is impossible during real-time clinical monitoring.

Unidirectional models such as LSTM and GRU are more appropriate for deployment because they can generate predictions immediately as new patient data becomes available. This allows continuous monitoring and timely clinical alerts, which are essential in critical care settings.

Although Bi-LSTM models remain valuable for retrospective analysis and research applications, their dependency on future information limits their use in real-world early warning systems.

Section 4: Transformers and Clinical Language Understanding

Transfer Learning with ClinicalBERT

Training a language model from scratch for healthcare applications requires extremely large datasets and computational resources. Medical language contains specialised terminology, abbreviations, and contextual relationships that are difficult to learn from small datasets alone.

To address this challenge, this project used ClinicalBERT, a Transformer-based language model pre-trained on more than two million clinical notes from the MIMIC-III database. During pre-training, the model learned medical vocabulary, clinical relationships, and contextual language patterns using masked language modelling.

Transfer learning allows the model to apply this previously learned medical knowledge to a new prediction task. For example, ClinicalBERT already understands that terms such as “sepsis,” “hypotension,” low SpO₂ values, and poor Glasgow Coma Scale scores are associated with critical patient conditions. Fine-tuning then adapts this existing knowledge specifically for mortality prediction.

Without transfer learning, training an equivalent language model from scratch would require far larger datasets and significantly greater computational cost.

Advantages of the Self-Attention Mechanism

Traditional sequence models such as LSTMs process text one token at a time from left to right. As sequences become longer, earlier information must be compressed into a fixed hidden representation, making long-range relationships difficult to capture.

Transformers solve this limitation using self-attention. Instead of processing words sequentially, self-attention allows every token in the sequence to interact directly with every other token simultaneously. The model learns attention weights that determine which words are most important for understanding the clinical note.

This mechanism closely resembles how clinicians read medical notes. Rather than analysing every word equally, physicians focus immediately on clinically significant terms such as “intubated,” “hypotension,” “sepsis,” or “low oxygen saturation.” Self-attention gives the Transformer a similar ability to prioritise the most informative parts of the text.

Because clinical meaning is often distributed across multiple parts of a note, self-attention is particularly effective for healthcare NLP tasks.

Insights from Attention Heatmaps

Attention visualisation was used to understand which terms ClinicalBERT focused on when making predictions. Attention weights from the final Transformer layer were extracted and analysed to identify clinically important tokens.

The frozen ClinicalBERT model focused strongly on terms related to oxygen saturation, Glasgow Coma Scale measurements, monitoring status, and admission details. These features are clinically meaningful because they are commonly associated with patient deterioration risk.

The fully fine-tuned model showed a different attention pattern. It focused more heavily on admission source information such as “recovery room” and on numerical identifiers related to diagnosis codes and vital signs. This indicates that fine-tuning allowed the model to adapt its attention toward features specifically relevant to mortality prediction.

Importantly, both models appeared to focus on medically meaningful information rather than unrelated correlations. This interpretability is one of the major advantages of Transformer-based models in healthcare. Unlike traditional deep learning models, attention visualisation can provide clinicians with insight into why a particular prediction was generated, improving transparency and trust.

Frozen Fine-Tuning vs Full Fine-Tuning

Two ClinicalBERT training strategies were evaluated in this project: frozen fine-tuning and full fine-tuning.

In frozen fine-tuning, all Transformer layers remain fixed and only the classification head is trained. This approach achieved very high Accuracy (0.9017) but extremely poor Recall (0.0952). The model largely predicted patients as survivors and failed to identify most high-risk cases.

In full fine-tuning, all 110 million model parameters were updated during training. Although this required significantly longer training time, the model achieved much stronger Recall (0.6857) and improved clinical sensitivity. The model became better at identifying deteriorating patients, even though this reduced overall Accuracy due to increased false positives.

Frozen fine-tuning is useful when datasets are very small or computational resources are limited. Full fine-tuning is preferable when sufficient training data and GPU resources are available because it allows the model to adapt more effectively to the target healthcare task.

Importance of Positional Encoding

Unlike LSTMs, Transformers process all tokens simultaneously rather than sequentially. Without additional positional information, the model would not understand word order within the sentence.

Positional encoding solves this problem by adding location information to token embeddings before they enter the Transformer layers. This allows the model to distinguish between phrases where the same words appear in different orders.

In clinical notes, sequence order is important because early observations, admission details, and later monitoring updates may carry different levels of clinical significance. Positional encoding helps the model preserve this structure while still benefiting from parallel processing.

Scalability for Hospital Deployment

One of the biggest advantages of Transformer architectures is parallelisation. Unlike recurrent models, which process sequences step by step, Transformers process all tokens simultaneously. This makes them highly efficient on modern GPU hardware.

For hospital-scale systems handling thousands of clinical notes daily, this efficiency becomes extremely important. Sequential models such as LSTMs may introduce delays when processing large volumes of data, while Transformer-based models can analyse many patient notes in parallel batches with very low latency.

This scalability makes ClinicalBERT and similar Transformer architectures strong candidates for large-scale healthcare AI deployment, provided sufficient computational infrastructure is available.

Generation 3 Results Summary

Model	Accuracy	Precision	Recall ★	F1-Score	Train
ClinicalBERT (Frozen)	0.9017	0.3030	0.0952	0.1449	279.2s
ClinicalBERT (Full)	0.7067	0.1841	0.6857	0.2903	645.8s

Final Deployment Recommendation

Among all evaluated models, the GRU was selected as the best option for real-time ICU deployment, while the fully fine-tuned ClinicalBERT model was considered more suitable for offline clinical risk analysis.

The GRU achieved the highest Recall score (0.7799) among all deployable models while maintaining fast training and low inference latency. Unlike a traditional DNN that only observes a single patient snapshot, the GRU captures temporal changes in vital signs over time, making it more suitable for monitoring patient deterioration. Its lightweight architecture also makes it practical for continuous hospital monitoring systems.

The DNN with Adam optimiser achieved the strongest F1-Score in the first generation of experiments and can still serve as a rapid triage model when only static patient measurements are available. In contrast, the fully fine-tuned ClinicalBERT model produced strong Recall

performance but required significantly higher computational resources and longer training time. Because of this, it is more appropriate as a secondary clinical review system rather than a primary bedside alert engine.

Although the Bidirectional LSTM achieved strong offline performance, it cannot realistically operate in live ICU monitoring because it depends on future timesteps that are unavailable in real-time settings. The frozen ClinicalBERT model also proved unsuitable for deployment due to extremely low Recall, meaning it failed to identify most high-risk patients.

Ethical and Clinical Considerations

Several ethical concerns must be addressed before deploying AI systems in healthcare environments.

Bias and Fairness

Clinical datasets may contain demographic imbalance across ethnicity, gender, or age groups. If certain populations are underrepresented during training, the model may perform worse for those patients, potentially increasing healthcare inequality. Proper subgroup evaluation and fairness auditing are therefore essential before deployment.

Privacy and Patient Data Protection

Clinical records contain highly sensitive patient information. Any real-world deployment must follow strict privacy regulations, including secure storage, anonymisation, controlled access, and institutional approval for model training and usage.

Human Oversight

AI systems should support clinicians rather than replace them. Every high-risk prediction should still be reviewed by healthcare professionals. Human oversight is critical to prevent harmful decisions caused by incorrect predictions or excessive false alarms.

Expanding the System with Medical Imaging

The system could be extended into a multimodal clinical AI architecture by combining clinical notes with chest X-ray analysis.

In such a setup, chest X-ray images would first be processed using a CNN-based image encoder such as ResNet or DenseNet. At the same time, clinical narratives would be processed using ClinicalBERT to extract textual representations. The outputs from both models would then be fused together through a shared classification layer or attention-based fusion module.

This design mirrors how physicians combine imaging findings with written clinical observations before making decisions. By integrating both visual and textual information, the system could potentially achieve more accurate and clinically reliable predictions.

Unified Model Comparison Table

Model	Accuracy	Precision	Recall	F1-Score	Train Time
DNN (Adam)	0.8618	0.3317	0.5681	0.4189	—
DNN (SGD)	0.8301	0.2921	0.6583	0.4046	—
LSTM	0.7721	0.2445	0.7652	0.3706	59.8s
GRU ★	0.7735	0.2482	0.7799	0.3765	74.7s
Bi-LSTM	0.7820	0.2523	0.7568	0.3784	97.2s
ClinicalBERT (Frozen)	0.9017	0.3030	0.0952	0.1449	279.2s
ClinicalBERT (Full)	0.7067	0.1841	0.6857	0.2903	645.8s